```java
public class ReverseWord {

    // Function to reverse a word using ListStack
    public static String reverseWord(String word) {
        ListStack stack = new ListStack();

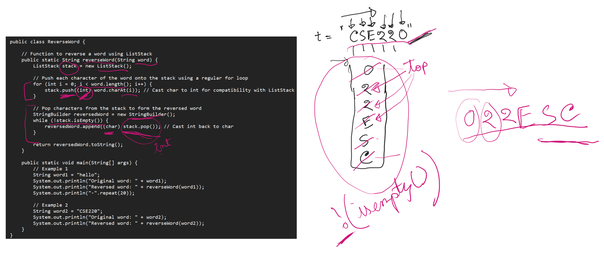
        // Push each character of the word onto the stack using a regular for loop
        for (int i = 0; i < word.length(); i++) {
            stack.push((int) word.charAt(i)); // Cast char to int for compatibility with ListStack
        }

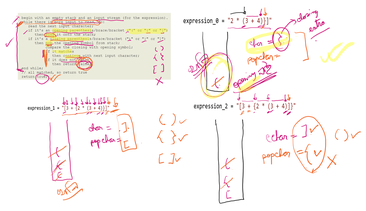
        // Pop characters from the stack to form the reversed word
        StringBuilder reversedWord = new StringBuilder();
        while (!stack.isEmpty()) {
            reversedWord.append((char) stack.pop()); // Cast int back to char
        }

        return reversedWord.toString();
    }

    public static void main(String[] args) {
        // Example 1
        String word1 = "hello";
        System.out.println("Original word: " + word1);
        System.out.println("Reversed word: " + reverseWord(word1));
        System.out.println("-".repeat(20));

        // Example 2
        String word2 = "CSE220";
        System.out.println("Original word: " + word2);
        System.out.println("Reversed word: " + reverseWord(word2));
    }
}
```






```java
public class CheckBalancedParentheses {

    public static void checkBalancedParentheses(String expression) {
        ListStack stack = new ListStack(); // Initialize a stack to hold opening brackets

        // Use a regular for loop instead of enhanced for loop
        for (int i = 0; i < expression.length(); i++) {
            char currentChar = expression.charAt(i);

            // Push opening brackets onto the stack
            if (currentChar == '(' || currentChar == '{' || currentChar == '[') {
                stack.push(currentChar);
            }
            // Check matching pairs for closing brackets
            else if (currentChar == ')' || currentChar == '}' || currentChar == ']') {
                if (stack.isEmpty()) {
                    System.out.println("Error: Extra closing bracket found");
                    return;
                }
                char lastOpening = (char) stack.pop();
                if ((lastOpening == '(' && currentChar != ')') ||
                    (lastOpening == '{' && currentChar != '}') ||
                    (lastOpening == '[' && currentChar != ']')) {
                    System.out.println("Error: Mismatched brackets");
                    return;
                }
            }
        }

        // If the stack is empty, all brackets were matched correctly
        if (stack.isEmpty()) {
            System.out.println("Ok, Expression is balanced");
        } else {
            System.out.println("Error: Extra opening brackets left");
        }
    }
```

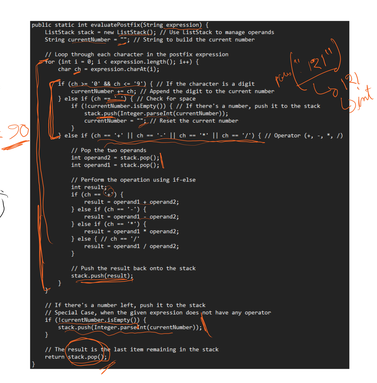
Postfix Expression Evaluation

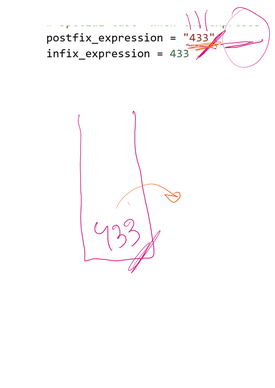
Postfix (Reverse Polish Notation):

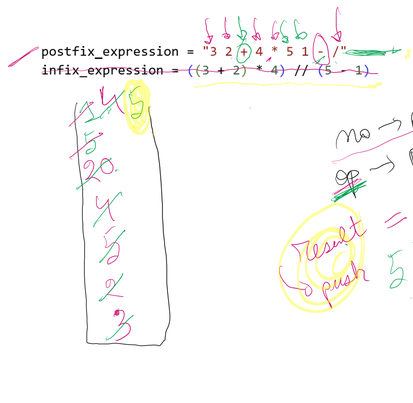
- Postfix eliminates the need for parentheses and precedence rules.
- Example:
  - Infix: 3 + 5 * 9
  - Postfix: 3 5 9 * +
- More complex example:
  - Infix: (3 + 2) * (5 - 1)
  - Postfix: 3 2 + 4 * 5 1 - /

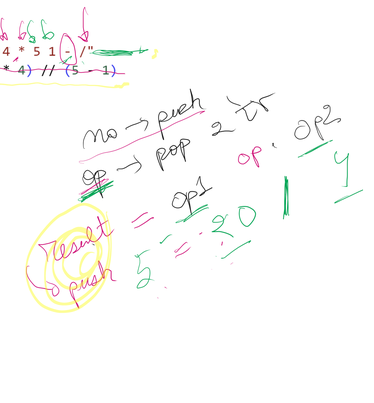
Using a Stack for Post-fix Evaluation

```
begin with an empty stack and an input stream (for the expression).
while there is more input to read, do:
    read the next input symbol;
    if it's an operand, or
        then push it onto the stack;
    if it's an operator,
        then pop the operands from the stack;
             perform the operation on the operands;
             push the result;
end while;
// the answer of the expression is waiting for you in the stack:
pop the answer;
```









SUMMER2025 QUESTION:--ONLY STACK

Suppose you are developing a personal banking app that tracks daily account activity. Each transaction recorded in the app represents either a deposit (a positive integer) or a withdrawal (a negative integer). These transactions are stored in a **Stack**, with the most recent transaction on top.

You need to implement a function named **depositSum()** that takes two arguments:

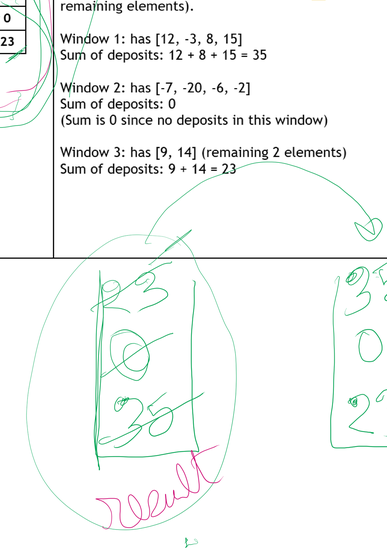
- a stack of transaction data and
- an integer representing the **window size**.

The function should compute the **sum of only the deposit values** within each consecutive window of the given size and return the modified stack containing these summed values.

If the last window contains fewer elements than the specified window size, it should still be processed using the remaining values. The most recent window's sum should appear at the top of the returned stack.
Modified stack means you must return the stack you received via the function parameter.

The Stack class is implemented for you. **its public instance methods are push(), pop(), peek(), and isEmpty()**. The isEmpty() method returns a boolean indicating whether the stack is empty. The peek() and pop() methods return null or None if the stack is empty (i.e., underflow condition). All other members of the Stack class are private and should not be accessed directly. You can only use the public instance methods provided. No need to consider the overflow exception for the Stack class. **You cannot use any data structure other than the stack. You can create instances of the stack as many times as you want.**

Input Stack	Modified Stack	Explanation
stack = Top → 12, -3, 8, 15, -7, -20, -6, -2, 9, 14 window = 4	stack = Top → 35, 0, 23	Since the window size is given as 4, each window will have 4 values (except possibly the last one, if there are fewer remaining elements). Window 1: has [12, -3, 8, 15] Sum of deposits: 12 + 8 + 15 = 35 Window 2: has [-7, -20, -6, -2] Sum of deposits: 0 (Sum is 0 since no deposits in this window) Window 3: has [9, 14] (remaining 2 elements) Sum of deposits: 9 + 14 = 23




```java
public static Stack depositSum(Stack st, int window) {

    Stack result = new Stack();

    // Process each window from the top of the stack
    while (!st.isEmpty()) {
        int depositSum = 0;
        int count = 0;

        // Process at most window elements
        while (count < window && !st.isEmpty()) {
            int val = st.pop();

            // Add only deposits
            if (val > 0) {
                depositSum += val;
            }

            count++;
        }

        // Store sum of current window
        result.push(depositSum);
    }

    // Put results back into the original stack
    while (!result.isEmpty()) {
        st.push(result.pop());
    }

    return st;
}
```